Unit II

2. Introduction Smart Phone Application Development and User Interface components, Widgets

2.1 Creating application template
2.2 Adding activity, intent, Intents and Intent Filters, Resources
2.3 System Permissions, services to application
2.4 Layouts, Recycler View, List View, Grid View and Web view
2.5 Input Controls: Buttons, Checkboxes, Radio Buttons, Toggle Buttons, Spinners, Input Events, Menus, Toast, Dialogs, Styles and Themes, Creating lists, and Custom lists

---

2.1 Creating application template:

Here's a concise **answer** on **Creating an Application Template in Android**:

Creating an Application Template in Android:

An **application template** is a basic structure or starting point for building an Android app. It includes default files, folders, and code necessary to kickstart app development.

Steps to Create an App Template in Android Studio:

1. Start a New Project:

Open **Android Studio** and select **"New Project."**

Choose a **template** (like Empty Activity, Navigation Drawer, etc.) depending on the app type.

2. Configure Project:

Enter **App Name**, **Package Name**, **Save Location**.

Select **Language** (Java/Kotlin) and **Minimum SDK version**.

3. Default Files Created:

`MainActivity.java` or `MainActivity.kt`: Main logic file.

`activity_main.xml`: UI layout file.

`AndroidManifest.xml`: Configuration and app declaration.

`build.gradle`: Handles dependencies and build configuration.

4. Project Structure:

Organized into **Java/Kotlin code**, **resources (res)**, **manifests**, and **Gradle scripts**.

5. Run the Template:

You can **run the app on an emulator or physical device** to see the default template in action.

This setup provides a solid foundation to start adding custom features and UI elements.

2.2 Adding activity, intent, Intents and Intent Filters, Resources:

Here’s a well-organized explanation on **Adding Activity, Intent, Intent Filters, and Resources** in Android:

1. Adding an Activity:

An **Activity** represents a single screen in an app.

To add a new activity:

Right-click on the `java` folder → New → Activity (choose type).

Android Studio generates a `.java` or `.kt` file and a corresponding XML layout.

Also updates `AndroidManifest.xml` with a new `<activity>` tag.

2. Intent:

Intent is a messaging object used to request an action from another component.

Two types:

Explicit Intent: Starts a specific activity (e.g., from MainActivity to SecondActivity).

Implicit Intent: Requests an action without naming the target (e.g., open browser, share image).

java

```
// Example of Explicit Intent
Intent i = new Intent(MainActivity.this, SecondActivity.class);
startActivity(i);
```

3. Intent Filters:

Declared in `AndroidManifest.xml`.

Used to specify which intents an activity can respond to (especially for **implicit intents**).

xml

```
<intent-filter>
    <action android:name="android.intent.action.VIEW" />
    <category android:name="android.intent.category.DEFAULT" />
    <data android:mimeType="text/plain" />
</intent-filter>
```

4. Resources:

Stored in the `res/` folder.

Types of resources:

layout/ – UI design XML files.

drawable/ – Images and graphic files.

values/ – Strings, colors, dimensions.

raw/ – Audio, video, and other raw files.

Use resources in code like:

java

```
String appName = getString(R.string.app_name);
```

2.3 System Permissions, services to application:

Here's a clear and concise explanation on **System Permissions** and **Services in Android Applications**:

1. System Permissions:

Android uses a **permission system** to protect user privacy and device security.

Apps must **declare permissions** in the `AndroidManifest.xml` to access sensitive features like camera, location, or internet.

Types of Permissions:

1. **Normal Permissions –** Low-risk (e.g., internet); automatically granted.
2. **Dangerous Permissions –** High-risk (e.g., camera, contacts); require **runtime approval** from the user.

Examples:

In `AndroidManifest.xml`:

```xml
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
```

At runtime (for dangerous permissions):

```java
ActivityCompat.requestPermissions(this, new
String[]{Manifest.permission.CAMERA}, 1);
```

2. Services in Android Applications:

A **Service** is a component that performs background tasks **without a user interface**.
Useful for tasks like music playback, downloading files, or syncing data.

Types of Services:

1. Started Service:

Starts with `startService()` and runs until stopped.

2. Bound Service:

Binds to another component with `bindService()` and runs as long as it's connected.

3. Foreground Service:

Runs with a notification, used for ongoing tasks like music or GPS tracking.

Example:

```java
Intent intent = new Intent(this, MyService.class);
startService(intent);
```

2.4 Layouts, Recycler View, List View, Grid View and Web view:

Here's a concise explanation on **Layouts, RecyclerView, ListView, GridView, and WebView** in Android:

1. Layouts:

Layouts define the **UI structure** of an Android app.
Common types:

LinearLayout: Arranges views in a single row or column.
RelativeLayout / ConstraintLayout: Positions views relative to each other or parent.
FrameLayout: Stacks views on top of each other.

2. RecyclerView:

An advanced and flexible view for displaying **large sets of data efficiently**.
Uses **ViewHolder pattern** for better performance.
Supports different layouts (list, grid, staggered) using **LayoutManager**.

java

```
RecyclerView recyclerView = findViewById(R.id.recyclerView);
recyclerView.setLayoutManager(new LinearLayoutManager(this));
```

3. ListView:

Displays data in a **scrollable vertical list**.
Requires an **Adapter** (like ArrayAdapter) to bind data to list items.
Less efficient compared to RecyclerView for large datasets.

java

```
ListView listView = findViewById(R.id.listView);
listView.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1,
dataList));
```

4. GridView:

Displays items in a **two-dimensional grid**.
Useful for showing images or product items.

java

```
GridView gridView = findViewById(R.id.gridView);
gridView.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1,
dataList));
```

5. WebView:

Allows you to **display web pages** inside an Android app.
Can load local or remote web content.

java

```
WebView webView = findViewById(R.id.webView);
webView.loadUrl("https://www.example.com");
```

2.5 Input Controls: Buttons, Checkboxes, Radio Buttons, Toggle Buttons, Spinners, Input Events, Menus, Toast, Dialogs, Styles and Themes, Creating lists, and Custom lists:

Here's a neatly summarized **answer** on key **UI Components and Concepts** in Android:

1. Input Controls:

Buttons: Trigger an action when clicked.
CheckBoxes: Allow multiple selections from a set of options.
Radio Buttons: Allow only one selection in a group.
ToggleButtons / Switch: Represent ON/OFF states.
Spinners: Drop-down list for single selection.

2. Input Events:

Handled using listeners like:

java

```
button.setOnClickListener(new View.OnClickListener() {
    public void onClick(View v) {
```

```
                // Action on click
            }
        });
```

3. Menus:

Add options in the app bar or as a context menu.

Options Menu: Common actions (e.g., settings).
Context Menu: Shown on long press.
Popup Menu: Anchored to a specific view.

4. Toast:

Simple message popup for a few seconds.

java

```
Toast.makeText(this, "Message Sent!", Toast.LENGTH_SHORT).show();
```

5. Dialogs:

Used to show alerts, confirmations, or input.
Types: **AlertDialog**, **DatePickerDialog**, **Custom Dialogs**.

6. Styles and Themes:

Style: Set of UI properties applied to views (like font, color).
Theme: Applies styles app-wide for a consistent look.

Defined in `res/values/styles.xml`.

7. Creating Lists and Custom Lists:

Default Lists: Use `ListView` with `ArrayAdapter` for simple lists.
Custom Lists: Use a custom layout and `BaseAdapter` or `RecyclerView.Adapter` for complex items (images, buttons, etc.).